

Lista: Modelos de Custo.

Nome: _____ **Matrícula:** _____

Exercícios baseados no capítulo 21 do livro Modern Programming Languages.

Exercício 01. Descubra o que o seguinte predicado misterioso faz e, em seguida, reimplemente-o da maneira mais eficiente possível. Note que se os parâmetros de `riddle` não estiverem totalmente instanciados, ele pode entrar em um loop infinito. Não se preocupe com esses casos: onde o predicado `riddle` dado entraria em um loop infinito, seu predicado mais eficiente pode fazer algo diferente.

```
riddle(X, _) :- length(X, XL), XL = 0.  
riddle(_, Y) :- length(Y, YL), YL = 0.
```

Exercício 02. Descubra o que o seguinte predicado misterioso faz e, em seguida, reimplemente-o sem usar `append`, da maneira mais eficiente possível.

```
mystery(Item, List) :- append([Item], List).
```

Exercício 03. Descubra o que o seguinte predicado misterioso faz e, em seguida, reimplemente-o sem usar `append`, da maneira mais eficiente possível. Note que se os parâmetros de `puzzle` não estiverem totalmente instanciados, ele pode entrar em um loop infinito. Não se preocupe com esses casos: onde o predicado `puzzle` dado entraria em um loop infinito, seu predicado mais eficiente pode fazer algo diferente.

```
puzzle(_, [], []).  
puzzle(E, [_|Tail], Result :-  
    puzzle(E, Tail, TailResult),  
    append(TailResult, [E], Result).
```

Exercício 04. Reescreva a função abaixo para que utilize recursão de cauda.

```
fun dividir nil = (nil, nil)  
| dividir [a] = ([a], nil)  
| dividir (h1::h2::t) = let
```

```
    val (esq, dir) = dividir t
  in
    (h1::esq, h2::dir)
  end;
```

Exercício 05. A função `prefixCopy x n`, escrita em ML, retorna uma lista igual a `x`, na qual os primeiros `n` elementos são cópias, enquanto todos os outros são compartilhados. Quando `n >= length(x)`, retorna uma cópia completa de `x`.

```
fun prefixCopy x 0 = x
  | prefixCopy nil _ = nil
  | prefixCopy (head::tail) n = head :: (prefixCopy tail (n-1));
```

Reescreva essa função para que use recursividade de cauda.

Exercício 06. Em muitos casos, a ordem mais eficiente para as condições em uma regra Prolog não pode ser determinada apenas olhando para a regra. Depende de outras partes do programa. Considere esses dois predicados `grandfather`:

```
grandfather1(X, Y) :- parent(X, Z), male(X), parent(Z, Y).
grandfather2(X, Y) :- male(X), parent(X, Z), parent(Z, Y).
```

- Suponha que o banco de dados tenha muitos fatos `parent` mas poucos fatos `male`. Qual predicado `grandfather` você prefere e por quê?
- Suponha que o banco de dados tenha muitos fatos `male` mas poucos fatos `parent`. Qual predicado `grandfather` você prefere e por quê?
- Considere os seguintes fatos: `male(joe)`, `male(bob)`, `parent(joe, bob)` e `parent(bob, alice)`. Dê uma ordem desses fatos para a qual a consulta `grandfather1(X, Y)` encontre a solução sem *backtracking*, enquanto `grandfather2(X, Y)` produza *backtracking*. Dê uma ordem diferente para a qual a consulta `grandfather1(X, Y)` produza *backtracking*, enquanto `grandfather2(X, Y)` encontra a solução sem *backtracking*. Em cada caso, explique os passos tomados pelo sistema Prolog para encontrar a solução, mostrando onde ocorre o *backtracking*.